

System Programming Project 2

담당 교수 : 김영재 교수님

이름 : 백서연

학번 : 20221197

1. 개발 목표

MyShell 프로젝트의 개발 목표는 사용자가 입력한 명령어를 해석하고 실행하는 **간단한 셸(Shell) 프로그램**을 구현하는 데 있습니다. 이 프로젝트는 Linux 운영체제의 프로세스 제어, 파이프라인, 시그널 처리, 백그라운드 프로세스 등의 개념을 실습을 통해 익히고, 이를 바탕으로 실제로 동작하는 커맨드 라인 인터페이스를 직접 만드는 것을 목표로 합니다.

프로젝트는 총 세 개의 단계로 나뉘며, 각 단계는 다음과 같은 기능 구현을 포함합니다:

1. Phase 1:

fork()와 exec()를 통해 명령어 실행을 위한 **자식 프로세스 생성 및 실행** 기능을 구현하고, signal을 활용하여 부모-자식 간의 시그널 처리를 다룹니다.

2. Phase 2:

'|' 연산자를 해석하여 **파이프라인(pipelining)** 기능을 구현합니다. 여러 명령어를 연결해 데이터를 전달하며 실행할 수 있도록 지원합니다.

3. Phase 3:

'&' 기호를 통해 **백그라운드(background) 실행** 기능을 추가합니다. 사용자가 기다리지 않고 다음 명령어를 바로 입력할 수 있게 만듭니다.

이러한 기능을 구현함으로써, 단순한 명령어 실행뿐 아니라 리눅스 시스템 프로그래밍의 핵심 요소들을 직접 체험하고 실습할 수 있으며, 실제 셸 프로그램의 작동 원리를 깊이 있게 이해하는 것을 최종적인 학습 목표로 삼습니다.

2. 개발 범위 및 내용

A. 개발 범위

1. Phase 1 – 기본 명령어 실행 및 시그널 처리

- 사용자가 입력한 명령어를 해석하여 ****자식 프로세스를 생성(fork)하고 명령을 실행(exec)****할 수 있습니다.
- ****내장 명령어(cd, exit, jobs, fg, bg 등)****도 처리할 수 있으며, 잘못된 명령어에 대해서는 적절한 오류 메시지를 출력합니다.

- 자식 프로세스 종료 시, ****시그널 핸들러(SIGCHLD 등)****를 통해 부모 프로세스가 이를 감지하고 적절히 처리하게 됩니다.

✅ **결과:** 기본적인 셸 명령 실행이 가능해지고, 자식 프로세스의 생성/종료를 통해 단일 명령어 처리 셸 완성.

2. Phase 2 – 파이프라인 기능 구현

- 사용자가 입력한 명령어 중 | 기호를 기준으로 **여러 개의 명령어를 파이프**로 연결하여 처리할 수 있습니다.
- 명령어 사이에 파이프를 설정하여 **출력을 다음 명령어의 입력으로 전달**하는 기능을 수행하며, **두 개 이상의 파이프**도 처리할 수 있습니다.

✅ **결과:** ls -al | grep txt | sort와 같은 **복합 명령어 처리**가 가능해지며, 데이터 흐름 기반 명령 실행이 구현됩니다.

3. Phase 3 – 백그라운드 명령어 실행

- 명령어 끝에 &가 포함된 경우, 해당 명령어를 **백그라운드에서 실행**되도록 처리합니다.
- 이때 부모 프로세스는 기다리지 않고 **즉시 다음 명령을 입력할 수 있도록 프롬프트를 반환**합니다.
- 백그라운드에서 실행 중인 작업은 jobs, fg, bg 등의 명령으로 **관리**할 수 있습니다.

✅ **결과:** 여러 작업을 병렬적으로 실행할 수 있게 되며, **멀티태스킹 환경의 셸** 기능을 완성합니다.

B. 개발 내용

Phase 1 (fork & signal)

- 사용자가 입력한 명령어를 처리할 때 fork()를 호출하여 **자식 프로세스**를 생성하고, 자식 프로세스에서 execvp()로 해당 명령을 실행합니다.

- 자식은 **자신만의 프로세스 그룹을 설정(setpgid)**하며, 부모는 이를 기반으로 **포그라운드/백그라운드 상태에 따라 제어**합니다.
 - 부모 프로세스는 SIGCHLD 시그널을 핸들링하여 **자식 프로세스의 종료나 정지 여부를 감지**하고, job 테이블에서 해당 정보를 업데이트하거나 삭제합니다.
 - SIGINT(Ctrl-C), SIGTSTP(Ctrl-Z)는 포그라운드 job의 프로세스 그룹 전체에 전달되어 중단 혹은 종료되도록 합니다. 이를 통해 사용자는 일반 셸처럼 명령어 실행 중 중단/정지/재시작이 가능합니다.
-

Phase 2 (pipelining)

- | 문자를 기준으로 명령어를 분리한 후, 각 명령어마다 별도의 프로세스를 생성합니다.
 - 각 프로세스는 이전 명령어의 출력을 다음 명령어의 입력으로 연결되도록 **pipe()와 dup2()를 이용해 파이프라인을 구성**합니다.
 - 첫 번째 명령어는 표준 입력을 사용하고, 중간 명령어는 **입력과 출력을 모두 리디렉션**, 마지막 명령어는 표준 출력을 사용합니다.
 - **명령어 개수에 따라 파이프를 동적으로 생성하고 처리**하며, 자식 프로세스는 실행 후 종료되며, 부모 프로세스는 모든 파이프라인 자식 프로세스가 끝날 때까지 기다립니다.
 - 포그라운드/백그라운드 여부(&)도 파이프라인 전체에 적용되어 처리됩니다.
-

Phase 3 (background process)

- 명령어 끝에 &가 있을 경우, 해당 프로세스를 **백그라운드 프로세스로 인식**하고, 부모는 기다리지 않고 즉시 프롬프트를 반환합니다.
- 백그라운드 프로세스는 **별도의 job으로 job 테이블에 등록**되며, 사용자는 jobs, fg, bg, kill 등의 내장 명령어로 해당 프로세스를 관리할 수 있습니다.
- job은 job id(jid)와 process group id(pgid)를 기준으로 구분되며, fg/bg 전환 시 해당 프로세스 그룹에 SIGCONT 시그널을 보내 실행을 재개합니다.

C. 개발 방법

✅ Phase 1 – fork()와 signal 구현

1. 자식 프로세스 생성 (fork)

- eval() 함수 내에서 사용자가 입력한 명령어에 대해 fork()를 호출하여 **자식 프로세스를 생성**.
- 자식 프로세스는 setpgid(0, 0)을 통해 **고유한 프로세스 그룹**을 구성.
- 부모는 setpgid(pid, pid)로 자식의 프로세스 그룹을 설정하고, waitpid()로 포그라운드 job이 끝날 때까지 대기.

2. 시그널 핸들링 추가

- sigchld_handler() : SIGCHLD를 받아 **자식 종료/정지 여부 감지**, waitpid()와 WNOHANG | WUNTRACED 사용.
- sigint_handler() : 포그라운드 job에 SIGINT 전달 (Ctrl-C).
- sigtstp_handler() : 포그라운드 job에 SIGTSTP 전달 (Ctrl-Z).
- 위 시그널 핸들러들은 main() 함수 내에서 sigaction()을 통해 등록.

3. Job 테이블 관리 구조체 추가

- job_t 구조체 정의: jid, pgid, state, cmdline 등 포함.
- jobs[] 배열로 최대 MAXJOBS개의 job을 관리.
- 관련 함수: addjob(), deletejob(), fgjob(), listjobs() 등.

✅ Phase 2 – pipelining (|) 구현

1. eval_pipe() 함수 추가

- |를 기준으로 명령어 분리 (예: strtok으로 각 명령어 분해 후 trim()으로 앞뒤 공백 제거).
- 명령어 수만큼 루프를 돌며 pipe()와 dup2()로 입출력 연결.

2. 자식 프로세스에서 파이프 설정

- 첫 명령어: 표준입력 사용.
- 중간 명령어: dup2를 이용해 stdin, stdout 재설정.
- 마지막 명령어: 표준출력 사용.

3. 파이프라인 프로세스 그룹 설정

- 모든 파이프라인 프로세스를 같은 pgid로 설정.
- 포그라운드/백그라운드 여부 판단하여 job 테이블에 등록하거나 대기.

✅ Phase 3 – background process (&) 구현

1. parseline() 함수 수정

- 명령어 마지막 토큰이 &일 경우, 해당 명령을 **백그라운드 명령어로 판단**하고 플래그(bg) 반환.

2. 백그라운드 실행 시 처리

- eval() 또는 eval_pipe()에서 bg == 1일 경우 waitpid()를 호출하지 않고 바로 prompt 반환.
- job 상태는 BG로 등록되며, jobs[]에 추가.
- 사용자에게 [jid] pid 출력.

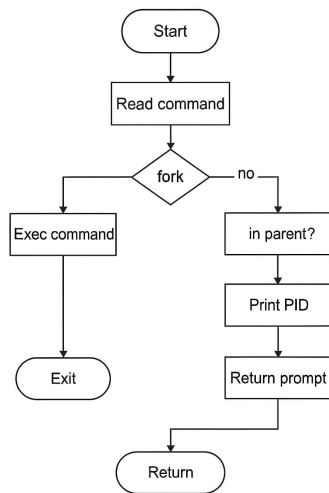
3. 내장 명령어 추가 (builtin_command)

- jobs → 현재 백그라운드/중지된 job 목록 출력.
- fg %jid, bg %jid → 포그라운드/백그라운드로 job 이동 (kill로 SIGCONT 전달).
- kill %jid → 해당 job 종료 (SIGTERM 전달).

3. 구현 결과

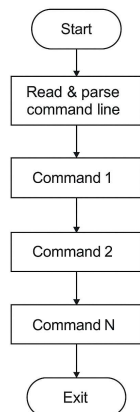
A. Flow Chart

1. Phase 1 (fork)



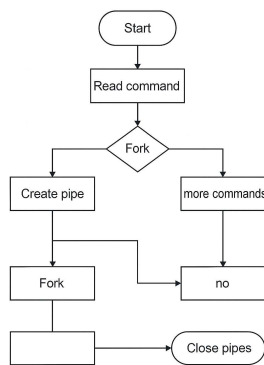
Phase 1: MyShell Fork

2. Phase 2 (pipeline)



Phase 2: MyShell Pipeline

3. Phase 3 (background)



Phase 2: Pipeline